NLP Midterm Cheat Sheet — CS2590 (Lectures 1–7) — Side A

L1: TEXT CLASSIFICATION

NLP Challenges

Ambiguity: syntactic ("I saw a man with a telescope") & semantic (polysemy/background (education/real)). **Situated understanding:** meaning depends on social/cultural context. **Data sparsity:** long tail of language, rare words, low-resource langs. **Bias:** models amplify biases from training data; lack common sense.

Penn Treebank: parse trees are broken into grammar rules such as **S→NP VP, NP→DT NN, VP→Vt NP / VP PP, PP→IN NP**, which are used to estimate PCFG rule probabilities from annotated data.

Example: *"The man sleeps"* breaks as **S→NP VP**, **NP→DT NN** ("The man"), **VP→Vi** ("sleeps"); tree probability is the product of the rule probabilities.

History

1940s–50s: Shannon's info theory, per-word log-likelihood, Shannon Game. **1950s–80s:** Chomsky, rule-based ("Colorless green ideas sleep furiously" — grammaticality $\neq$ probability). **1990s:** statistical NLP, Penn Treebank, PCFGs. **2010s+:** deep learning, large-scale neural models.

Classification Setup

Input x (text), output $y \in C$ (finite label set), classifier $f(x) = y$. **Binary** (2 classes) vs. **multiclass** (>2). **Supervised learning:** train/dev/test splits. Dev for hyperparameter tuning, test for final eval. Sensitive to distribution shift.

Evaluation Metrics

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = \frac{2PR}{P + R}$$

Accuracy = correct/total — misleading w/ class imbalance (e.g., 99% non-spam → always predict non-spam = 99% acc but 0% recall on spam). F1 is harmonic mean of P and R.

Feature Representation

Bag-of-Words (BoW): $\phi(x) \in \mathbb{R}^{|V|}$, each dim = word count or binary. Ignores word order & semantic similarity. Longer docs have larger feature vectors → more influence on learning.

Logistic Regression

Binary (sigmoid):

$$P(y{=}1 \mid x, w) = \sigma(w \cdot \phi(x)) = \frac{1}{1 + e^{-w \cdot \phi(x)}}$$

Multiclass (softmax):

$$P(y \mid x, w) = \frac{e^{w_y \cdot \phi(x)}}{\sum_{y'} e^{w_{y'} \cdot \phi(x)}} \quad \hat{y} = \arg\max_y w_y \cdot \phi(x)$$

Loss: NLL (cross-entropy): $\mathcal{L}(w) = -\sum_i \log P(y_i \mid x_i, w)$. NLL is **convex** → local min = global min. **SGD:** $w \leftarrow w - \eta \nabla_w \mathcal{L}$. **L2 reg:** add $\lambda \|w\|^2$ → penalizes large weights → prevents overfitting.

Perceptron

Predict: $\hat{y} = \text{sign}(w \cdot \phi(x))$. **Update:** only on mistakes: $w \leftarrow w \pm \phi(x)$ (add for false neg, subtract for false pos). **Convergence:** guaranteed if linearly separable. Has mistake bound. **vs. LR:** no probabilistic output; updates only on errors; LR updates on every example.

Feedforward Neural Networks

$$P(y \mid x) = \text{softmax}(W \cdot g(V \cdot \phi(x) + b_1) + b_2)$$

Activations: Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$, Tanh, **ReLU** $\max(0, z)$. **Key:** FFNN w/o non-linearity = single linear transform = logistic regression. **XOR:** not linearly separable; NN with hidden layer solves it. **Backprop:** chain rule for efficient gradient computation through layers.

Regularization Techniques

L2 reg: penalize large weights via $\lambda \|w\|^2$. **Dropout:** during training, randomly zero out neurons with probability p. Forces redundant reps, reduces co-adaptation. At test time, scale by $(1 - p)$ or use inverted dropout.

L2: WORD EMBEDDINGS

Motivation & Distributional Semantics

One-hot: $\mathbb{R}^{|V|}$, sparse, no similarity ($w_i^T w_j = 0$ for $i \neq j$). **Distributional hypothesis:** "know a word by the company it keeps" (Firth). Goal: dense, low-dim vectors $\mathbb{R}^d$ ($d \ll |V|$). **Brown clusters:** hierarchical agglomerative clustering by co-occurrence; hard clustering.

Word2Vec — CBOW & Skip-Gram

CBOW: predict center word from context. **Skip-gram:** predict context from center word.

$$P(c \mid w) = \frac{e^{v_c^T v_w}}{\sum_{c'} e^{v_{c'}^T v_w}}$$

Two vector sets: word embeddings v_w and context embeddings v_c. Total params: $2 \times |V| \times d$.

Training Efficiency

Hierarchical softmax: binary tree over vocab → $O(\log |V|)$ vs $O(|V|)$. **Negative sampling:** for each positive (w, c), sample k negative pairs. Binary classification:

$$\mathcal{L} = \log \sigma(v_c^T v_w) + \sum_{i=1}^{k} \log \sigma(-v_{c_i}^T v_w)$$

Even if two words always co-occur, they may be sampled as negatives (depends on context window).

GloVe (Pennington 2014)

Global co-occurrence matrix X. Weighted least squares:

$$\mathcal{L} = \sum_{i,j} f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

$f(x)$ caps very frequent pairs (e.g., $f(x) = \min((x/x_{\max})^a, 1)$).

fastText (Bojanowski 2017)

Skip-gram on character n-grams. Word vec = sum of n-gram vecs. **Key:** generates embeddings for OOV words by composing known n-grams.

Evaluation & Bias

Intrinsic: word similarity (cosine sim), analogies ($\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$). **Extrinsic:** downstream task performance. **Bias:** embeddings capture societal biases (man→doctor, woman→nurse). **TF-IDF:** sparse, high-dim; $\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$.

DAN, ELMo, Contextualized Embeddings

DAN: avg word embeddings → FFN → classify. Simple, loses order. **ELMo:** contextualized from bidirectional LSTM — same word gets different vectors in different contexts (polysemy). Embeddings = function of entire input sentence.

L3: SEQUENCE MODELING

Sequence Labeling Tasks

POS tagging: assign grammatical category (noun, verb, adj, etc.) to each word. Same word can have multiple tags by context ("back" = adj/noun/adv/verb). **NER:** identify & classify named entities. **BIO scheme:** B-TYPE (beginning), I-TYPE (inside), O (outside).

Language Modeling

Assign probability to word sequences. Chain rule:

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1})$$

Noisy Channel Model

$$\hat{x} = \arg\max_x P(x \mid a) = \arg\max_x \underbrace{P(a \mid x)}_{\text{channel}} \cdot \underbrace{P(x)}_{\text{LM}}$$

Apps: ASR (a=audio, x=text), MT (a=source, x=target), grammar correction. $P(a \mid x)$ = acoustic/translation model, $P(x)$ = language model prior.

N-gram Language Models

Markov: $P(x_i \mid x_1, \ldots, x_{i-1}) \approx P(x_i \mid x_{i-k+1}, \ldots, x_{i-1})$. Estimate by counting n-gram occurrences. Memory $\propto V^N$ (exponential). **Add-one (Laplace) smoothing:** add 1 to all counts — overestimates rare events. **Perplexity:**

$$\text{PP}(x) = 2^{-\frac{1}{N}\sum_{i=1}^{N} \log_2 P(x_i | x_{<i})}$$

Lower = better. Only comparable between models with same vocabulary.

Intuition: perplexity = average branching factor: how many next-token choices the model is effectively unsure among. Lower = less surprised.

Neural Language Models

Feedforward neural LM: concat embeddings of prev n words → hidden layers → softmax over $|V|$. **Limits:** fixed context size; no weight sharing across positions. **DAN:** avg embeddings → FFN; loses word order entirely.

Recurrent Neural Networks (RNNs)

$$h_t = g(W h_{t-1} + U x_t + b) \qquad y_t = \text{softmax}(V h_t + b_o)$$

W, U, b **shared across all timesteps**. **Modes: Transducer** — output per step (POS, LM). **Acceptor** — single output from final h_n (classification). **Limits:** (1) sequential, no parallelism; (2) vanishing/exploding gradients → hard to learn long-range deps.

LSTM (Long Short-Term Memory)

Uses gates (input i_t, forget f_t, output o_t) with sigmoid to control info flow. Cell state c_t carries long-range info:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad h_t = o_t \odot \tanh(c_t)$$

Mitigates vanishing gradients. **Other solutions:** GRU (simpler gating), attention, gradient clipping, skip connections.

Seq2Seq Models

Encoder: RNN → final hidden state = context vector c. **Decoder:** RNN initialized with c → generates output tokens autoregressively. **Teacher forcing:** feed ground-truth prev token during training (stabilizes). **Scheduled sampling:** gradually transition from ground-truth to model's own predictions (bridges train-test gap).

Exposure bias: train sees gold previous tokens; test sees the model's own outputs, so mistakes can compound.

Attention Mechanism

Motivation: fixed-size context vector is bottleneck for long sequences. Decoder attends to ALL encoder hidden states at each step.

$$e_{ij} = f(h_i^{\text{dec}}, h_j^{\text{enc}}) \quad \alpha_{ij} = \frac{e^{e_{ij}}}{\sum_k e^{e_{ik}}} \quad c_i = \sum_j \alpha_{ij} h_j^{\text{enc}}$$

$$P(y_i) = \text{softmax}(W[c_i; h_i^{\text{dec}}])$$

Score Function	Formula
Additive (Bahdanau)	$v^T \tanh(W_1 h_i + W_2 h_j)$
Dot-product (Luong)	$h_i^T h_j$
Bilinear (Luong)	$h_i^T W h_j$

Additive: most expressive (learnable non-linear combination); slower. **Dot-product:** fastest, simplest; assumes q,k same dim. **Bilinear:** intermediate — adds a learned interaction matrix.

L3/L4: TRANSFORMER

Advantages over RNNs

(1) **Parallelism:** all tokens processed simultaneously. (2) **O(1) path length** between any two positions (vs. O(n) for RNNs) → easier to learn long-range deps.

Self-Attention (Scaled Dot-Product)

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Intuition: like a fuzzy hash table — each query matches every key to varying degrees, then retrieves a weighted blend of values. Lets every token look at every other token for context.

Scaling by $\sqrt{d_k}$ prevents large dot products → softmax saturation → vanishingly small gradients. For large d_k, dot products grow $\propto d_k$ in variance.

Multi-Head Attention

$$\text{MH}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attn}(QW_i^Q, KW_i^K, VW_i^V)$$

Each head has **separate** W_i^Q, W_i^K, W_i^V matrices. Allows attending to different representation subspaces. **Bug:** if same Q,K for all heads → identical attention maps → defeats purpose.

Intuition: each head tracks a different relationship type (e.g., subject, modifier, coreference). Splitting into heads avoids one softmax having to express a single dominant pattern.

Positional Embeddings

Transformer has no recurrence → no inherent sense of order. Add positional info to input embeddings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

RoPE: rotates Q,K vectors by position angle. Used in modern LLMs (LLaMA, etc.). More effective than additive positional embeddings for extrapolation.

Sinusoidal: fixed, deterministic, generalizes to unseen lengths in theory. **RoPE:** encodes *relative* position via Q,K rotation; extrapolates better than absolute embeddings; applied in each attention layer, not just input.

Encoder Layer

Self-Attn → Add & LayerNorm → FFN → Add & LayerNorm. FFN:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \ \ \text{(two linear layers + ReLU)}$$

Residual connections: $\text{out} = \text{LayerNorm}(x + \text{Sublayer}(x))$. Helps gradient flow in deep networks. **LayerNorm:** normalizes to zero mean, unit variance per layer → stabilizes training.

Decoder Layer

Three sub-layers: (1) **Masked self-attention** — prevents attending to future positions (causal mask: set future entries to $-\infty$ before softmax). Ensures prediction at position i depends only on positions $< i$. (2) **Encoder-decoder attention** — Q from decoder, K and V from encoder output. (3) **FFN**. Each with residual + LayerNorm.

Decoder-Only Architecture (GPT-style)

Stack of masked self-attention layers only (no encoder). Autoregressive: generates one token at a time. Final layer: linear projection + softmax over vocabulary. Used by GPT-2/3, LLaMA.

Efficiency Concerns / Modifications to Attention

Self-attn complexity: $O(n^2 \cdot d)$ where n = seq length. where d = dimension length. **KV cache:** during inference, cache K,V for all past tokens → avoid recomputation. **Grouped Query Attention (GQA):** share K,V caches across attention heads; each group of query heads shares one K,V (intermediate between multi-head and multi-query). **Multi-Head Latent Attention:** map input to smaller latent vectors c_t^Q, c_t^{KV} before mapping to K,V,Q; compresses to bottleneck first → less memory.

NLP Midterm Cheat Sheet — CS2590 (Lectures 1–7) — Side B

L4/L5: TOKENIZATION & DECODING

Tokenization

Pre-tokenization: regex-based splitting by spaces/punctuation (prevents cross-word merges like "ing to"). **BPE (Byte-Pair Encoding):** (1) Init vocab with all bytes/chars. (2) Count adjacent pair frequencies. (3) Merge most frequent pair → new token. (4) Repeat until desired vocab size. Open vocabulary: handles OOV via subword decomposition. Shares representations between related words ("love", "loving"). **Unigram LM tokenization:** start with large vocab, iteratively prune tokens with lowest likelihood impact. **Encoding:** text → token IDs. **Decoding:** token IDs → text.

BPE vs Unigram: BPE greedily builds vocabulary by merges; Unigram starts large, then deletes unlikely pieces. Constructive vs. pruning.

Decoding Strategies

Autoregressive generation: generate one token at a time conditioned on all previous. Complexity: $O(k \cdot p \cdot L + k^2 \cdot L)$ for p prompt tokens, k generated tokens, L layers.

Method	How It Works	Pros/Cons
Greedy	$\hat{x}_t = \arg\max_{x_t} P(x_t \mid x_{<t})$	Fast; locally optimal ≠ globally optimal
Beam	Keep top-K hypotheses; expand & prune at each step by cumulative log-prob	Better quality; can be repetitive/generic (degeneration)
Sampling	Draw $x_t \sim P(x_t \mid x_{<t})$	Diverse; may produce incoherent text
Top-K	Sample from K most probable tokens only	Balances quality and diversity
Top-P (Nucleus)	Sample from smallest set with $\sum P \geq p$	Adaptive truncation; adjusts to distribution shape

Temperature T: $P'(x) \propto P(x)^{1/T}$. $T > 1 \to$ flatter (more random). $T < 1 \to$ sharper (more deterministic). $T = 1 \to$ original distribution. $T \to 0 \to$ greedy. $T \to \infty \to$ uniform.

Top-K vs Top-P: Top-K uses a fixed number of candidates; Top-P uses a variable set covering enough probability mass, so it adapts to sharp vs. flat distributions.

Degeneration

Beam search can yield repetitive, generic text because models are **locally normalized**: each next token can be likely while the full sequence is bad. Analogy: all-heads is the most likely coin-flip sequence, but atypical. Motivates sampling.

Advanced Decoding

Constrained decoding: structural (valid JSON, grammar) or lexical (force/ban words). **Contrastive decoding:** score $= \log P_{\text{large}}(x) - \log P_{\text{small}}(x) \to$ fluent and specific (amplifies what the large model knows that the small model does not). **DEXPERTS:** expert + anti-expert mixture for controlled generation (steers generation toward the expert's style and away from the anti-expert's). **Test-time scaling:** generate N candidates → verifier scores → select best (max) or weighted vote. **Speculative decoding:** small draft model proposes k tokens → large model verifies in one parallel pass (amortizes cost: one cheap sequential draft + one parallel verify pass). Speeds up inference without changing output distribution.

Hardware-Aware Optimizations

FlashAttention: fuses attention computation, reduces HBM↔SRAM memory transfers. Exact attention (not approximate). **PagedAttention:** manages KV cache in non-contiguous memory pages (like OS virtual memory) → reduces memory waste.

Intuition: FlashAttention avoids materializing huge attention matrices in memory; PagedAttention stores KV cache in chunks so memory packs more efficiently.

L6: PRE-TRAINING

LM Architecture Taxonomy

Type	Direction	Examples	Best For
Encoder-only (MLM)	Bidirectional	BERT, RoBERTa, DeBERTa	Classification, NER, QA
Decoder-only (Causal LM)	Left-to-right	GPT-2/3/4, LLaMA	Generation
Encoder-Decoder	Both	T5, BART, MASS	Translation, summarization

Intuition: encoder-only = understand, decoder-only = generate, encoder-decoder = read then write.

BERT

Encoder-only. **Base:** 12 layers, 768 hidden, 12 heads, 110M. **Large:** 24 layers, 1024 hidden, 16 heads, 340M. **Training objectives:** (1) **MLM:** mask 15% of tokens (80% [MASK], 10% random, 10% unchanged) → predict original. (2) **NSP:** given sentence A and B, predict if B is the actual next sentence. **Usage:** [CLS] token for sentence-level classification; token outputs for NER; predict start/end positions for extractive QA.

BERT Extensions

RoBERTa: more data, longer training, dynamic masking (re-mask each epoch), removes NSP, larger batches. **DeBERTa:** disentangled attention (separate content and position), enhanced mask decoder.

Encoder-Decoder Models

T5: text-to-text: every task is input string → output string; task prefix specifies what to do. Trained on C4 (Colossal Cleaned Common Crawl) with span corruption denoising. **BART:** denoising autoencoder — corrupt text (mask, delete, permute, rotate) → reconstruct. **MASS:** mask contiguous span in encoder → predict in decoder.

Paradigms for Using LMs

Pre-train + Fine-tune: pre-train on massive unlabeled corpus → fine-tune on task-specific labeled data. **ELMo:** contextualized embeddings from biLSTM (feature-based, not fine-tuned). **Prompting / In-context Learning (ICL):** frozen model + prompt with instructions ± few-shot examples. **Prompt Learning:** template $f_{\text{template}} : x \to x'$ with slot [Z]; $\hat{z} = \arg\max_z P(f_{\text{fill}}(x', z); M_\theta)$ (e.g. sentiment: "[X]. Overall, it was a [Z] movie"). **Instruction tuning:** fine-tune on diverse tasks formatted as instructions. ICL is sensitive to (1) example choice, (2) ordering, (3) format/template. LLaMA 2: SFT on smaller high-quality data. Issues: data quality, hallucinations, lack of creativity.

Building LMs — Key Stages

(1) Tokenizer building (BPE). (2) Pre-training (next-token prediction). (3) Instruction tuning (SFT). (4) RLHF alignment. **Training data principles:** scale + quality + diversity. Curation: dedup, safety filter, cleaning, heuristic filtering. Synthetic data increasingly used. Notable datasets: The Pile (825 GB), FineWeb (108 TB), Dolma (3T tokens).

Parameter-Efficient Fine-Tuning (PEFT)

Prefix tuning: learnable continuous prefix vectors are injected at **every transformer layer**, while **prompt tuning** adds soft prompt tokens only at the **input layer**. **BitFit:** fine-tune only bias terms (tiny fraction of params). **Adapters:** insert small FFN layers into each transformer layer; train only adapters. **LoRA (Low-Rank Adaptation):**

$$W' = W + \Delta W = W + BA \quad \text{where} \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d}, \ r \ll d$$

Only B and A are trained. Drastically reduces trainable parameters (e.g., $r = 8$ vs. $d = 4096$).

Intuition: freeze the big matrix and learn only a small low-rank correction, so adaptation is cheap.

L7: POST-TRAINING & RLHF

Goals of Post-Training

Align model to human preferences (helpful, harmless, honest). Distill reasoning ability. Extend context length. Post-training turns a base LM into a useful assistant.

RL Formalization & LLM Mapping

MDP comps: S states, $s_1 \in S$ initial state, A actions, $T : S \times A \to S$ transition, $r : S \times A \to \mathbb{R}$ reward. **Policy:** $\pi(a \mid s) = \mathbb{P}(A = a \mid S = s)$ = distribution over actions given state.

$$\tau = (a_1, s_1, \ldots, a_T, s_T) \qquad \max_\theta \ \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=1}^{T} r(s_t, a_t) \right]$$

LLM mapping: $s_t = (x_1, \ldots, x_{t-1})$, $a_t = x_t$, $T(s, a) = [s; a]$, policy $= p_\theta(x_t \mid x_{<t})$, trajectory = full generation $x_1, \ldots, x_T$. RL = choose token distribution so sampled completions get high total reward.

REINFORCE (Policy Gradient)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right) \left(\sum_{t=1}^{T} r(s_t, a_t) \right) \right]$$

Key idea: log-derivative trick turns gradient of expectation into expectation of gradient. **Monte Carlo:** sample N trajectories, form $\hat{g}$, update $\theta \leftarrow \theta + \alpha \hat{g}$.

LLM intuition: sample N generations, score each, multiply every token log-prob gradient by total reward; positive reward pushes sampled tokens up, negative reward pushes them down. **Issues:** high variance; credit assignment = late/sparse reward gives weak signal about which token mattered.

Reward Model Training

Why pairwise? Humans are noisy at absolute 1-10 scoring, more consistent at "which is better?". **Data:** (x, y_w, y_l) = prompt, winning output, losing output.

$$P(y_w \succ y_l) = \sigma\big(r_\theta([\vec{x}; \vec{y}_w]) - r_\theta([\vec{x}; \vec{y}_l])\big)$$

$$\mathcal{L}_r = -\mathbb{E}\left[\log \sigma\big(r_\theta([\vec{x}; \vec{y}_w]) - r_\theta([\vec{x}; \vec{y}_l])\big)\right]$$

Bradley-Terry: reward difference induces preference probability. **Intuition:** binary ranking loss raises winner score, lowers loser score. **Iterative:** e.g. Llama 2 used multiple rounds: train model/ranker -> sample -> annotate -> retrain; more data + bigger models usually improve RM quality.

RLHF Pipeline

RL objective: learn a policy π_θ that maximizes the **expected return**, i.e., the expected sum of rewards over a trajectory sampled from the policy.

$$\max_\theta \ \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=1}^{T} r(s_t, a_t) \right]$$

REINFORCE algorithm: 1) sample trajectories $\tau^1, \ldots, \tau^N$ from π_θ; 2) compute gradient estimate using total reward of each trajectory; 3) update policy by gradient ascent $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$; 4) repeat until convergence.

$$\nabla_\theta J(\theta) \approx \sum_{i=1}^{N} \left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) \right) \left(\sum_{t=1}^{T} r(s_t^i, a_t^i) \right)$$

Step 1 — Preference data: for each prompt, generate two outputs; human picks preferred one → dataset of (x, y_w, y_l) triplets. Pairwise comparison more reliable than absolute ratings.

Step 2 — Reward model: fit pairwise ranker r_θ. **Step 3 — optimize policy:** either PPO (explicit RM + RL) or DPO (direct preference loss, no separate RM).

PPO-based RLHF (4 models): uses a **policy** to generate responses, a **reward model** to score them, a **value model** to estimate advantages, and a frozen **reference policy** to apply a KL penalty and keep the policy close to the SFT model.

PPO steps: 1) sample prompts and generate responses from the current policy; 2) score each response with the reward model and add KL penalty against the reference policy; 3) use the value model to compute advantages; 4) update the policy with the PPO objective and update the value model, then repeat.

$$\max_\theta \ \mathbb{E}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)} \hat{A}(s, a) - \beta \, \mathrm{KL}(\pi_{\text{ref}}(\cdot \mid s), \pi_\theta(\cdot \mid s)) \right]$$

PPO

$$\arg\max_\theta \ E_\pi \left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)} \hat{A}(s, a) - \beta \, \mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s), \pi_\theta(\cdot \mid s)] \right]$$

Advantage: $A_\pi(s, a) = r(s, a) + V_\pi(T(s, a)) - V_\pi(s)$ = how much better action was than expected. **Critic:** learned value $V_\phi(s)$ estimates expected future reward, giving a baseline and lowering variance vs raw reward. **KL term:** keeps policy near previous/reference policy for stability - rubber band to SFT model.

4 models in memory: policy π_θ, value network V_ϕ, reward model r, reference policy π_{ref} (frozen SFT copy). **On-policy:** must sample fresh completions from current policy each iteration. **Intuition:** PPO = REINFORCE + critic baseline + trust-region/KL control; more hyperparams/engineering, but more stable.

DPO (2 models): uses only a trainable **policy** and a frozen **reference policy**; it does **not** require a separate reward model or value model.

DPO steps: 1) collect preference pairs (x, y_w, y_l) with winner and loser responses; 2) compute policy and reference log-probabilities for both responses; 3) optimize a preference loss that increases the winner's relative probability and decreases the loser's; 4) repeat over the dataset.

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right)$$

DPO (Direct Preference Optimization)

Motivation: PPO/RLHF is heavy: reward model training, RL instability, 4 models in memory. Can preferences be learned directly? **Starting point:** same KL-constrained RL objective. **Key insight:** solve for optimal policy, plug into Bradley-Terry; partition function $Z(\vec{x})$ cancels.

$$\mathcal{L}_{\text{DPO}}(\theta) = -E_{(\vec{x}, \vec{y}_w, \vec{y}_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(\vec{y}_w \mid \vec{x})}{\pi_{\text{ref}}(\vec{y}_w \mid \vec{x})} - \beta \log \frac{\pi_\theta(\vec{y}_l \mid \vec{x})}{\pi_{\text{ref}}(\vec{y}_l \mid \vec{x})} \right) \right]$$

Implicit reward: $\hat{r}(\vec{x}, \vec{y}) = \beta \log \frac{\pi_\theta(\vec{y} \mid \vec{x})}{\pi_{\text{ref}}(\vec{y} \mid \vec{x})}$, so no separate reward model. **2 models only:** trainable policy π_θ + frozen reference π_{ref}. **Off-policy:** trains on fixed preference dataset, no fresh rollout loop.

$$\nabla \mathcal{L} = -\beta E \left[\underbrace{\sigma(\hat{r}_l - \hat{r}_w)}_{\text{higher when wrong}} \left(\underbrace{\nabla \log \pi_\theta(\vec{y}_w \mid \vec{x})}_{\uparrow \text{preferred}} - \underbrace{\nabla \log \pi_\theta(\vec{y}_l \mid \vec{x})}_{\downarrow \text{dispreferred}} \right) \right]$$

Gradient intuition: β scales update / KL strength. $\sigma(\hat{r}_l - \hat{r}_w)$ is a per-example weight: big when model ranks loser too highly, small when it already gets pair correct. Push winner up, loser down.

Baselines & Variants

Best-of-N: sample N outputs from SFT, score with RM/verifier, pick best; no weight update, but inference-expensive. **Preference tuning:** SFT on preferred outputs only; simple, ignores negative signal from loser. **GRPO:** DeepSeek-R1-style variant; compute advantage relative to a group of sampled responses instead of a separate value network.

Post-Training Algorithm Comparison

Property	REINFORCE	PPO	DPO
Reward model	Required (ext.)	Required (ext.)	Implicit (no RM)
# models in memory	2 (policy+RM)	4 (policy,value,RM,ref)	2 (policy+ref)
On/Off-policy	On-policy	On-policy	Off-policy
Main signal	Raw return	Advantage + KL	Pairwise pref. loss
Variance	High	Lower (adv.)	Low (sup.-style)
Complexity	Simple	Complex (4 models, hypers)	Simple (single loss)
Stability	Unstable	More stable (KL + clipping)	Stable; may plateau
Sample efficiency	Low	Moderate	High (fixed data)

Alignment Issues

Reward hacking / length bias: RM can overvalue longer answers; older datasets saw large reward gains explained mainly by verbosity (~85% in some analyses). **Sycophancy:** model agrees with user's stated belief even when false; tends to rise with RL steps/model size. **Sandbagging:** weaker answers for prompts framing user as less educated. **Annotator disagreement:** preference labels are noisy and value-laden.

RL with Verifiers: for checkable tasks (math, code), use a verifier instead of a learned reward model to reduce reward hacking. Generate N candidates → verifier scores → pick best. Common in modern post-training pipelines.

KEY FORMULA REFERENCE

Name	Formula
Softmax	$P(y) = e^{z_y} / \sum_{y'} e^{z_{y'}}$
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$
NLL Loss	$\mathcal{L} = -\sum_i \log P(y_i \mid x_i)$
REINFORCE	$\nabla J = \mathbb{E}\left[\sum_t \nabla \log \pi_\theta(a_t \mid s_t) \cdot R(\tau)\right]$
Attn Weights	$\alpha_{ij} = \frac{e^{s_{ij}}}{\sum_k e^{s_{ik}}}$
Scaled Dot-Prod Attn	$\mathrm{softmax}(QK^T/\sqrt{d_k}) \cdot V$
PPO Advantage	$A_t = R_t - V_\phi(s_t)$
PPO KL Penalty	$\beta \, D_{KL}(\pi_\theta \| \pi_{\text{ref}})$
Chain Rule (LM)	$P(x) = \prod_i P(x_i \mid x_{<i})$
Perplexity	$\mathrm{PP} = 2^{-\frac{1}{N}\sum \log_2 P(x_i)}$
Temperature	$P'(x) \propto P(x)^{1/T}$
Noisy Channel	$\arg\max_x P(a \mid x) \cdot P(x)$
RNN Transition	$h_t = g(Wh_{t-1} + Ux_t + b)$
Bradley-Terry	$P(w \succ l) = \sigma(r(w) - r(l))$
LoRA	$W' = W + BA$
FFN	$\max(0, xW_1 + b_1)W_2 + b_2$
Skip-gram NS	$\log \sigma(v_c'^T v_w) + \sum \log \sigma(-v_{c_i}'^T v_w)$
GloVe	$\sum f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$
LSTM Forget Gate	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$
DPO Loss	$-\log \sigma(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)})$

ARCHITECTURE COMPARISON

Model	Var. Len?	Parallel?	Params Shared?	Long-Range?	Limitation
FFN	No	Yes	No	No	Fixed input
RNN	Yes	No	Yes (time)	Weak	Vanishing grad
LSTM/GRU	Yes	No	Yes	Better	Still sequential
Transformer	Yes	Yes	Yes (pos.)	Strong O(1)	$O(n^2)$ attn

KEY DISTINCTIONS

Encoder-only (BERT: bidirectional, classification) vs. **Decoder-only** (GPT: autoregressive, generation) vs. **Enc-Dec** (T5: both). **Static embeddings** (Word2Vec, GloVe: one vec per word) vs. **Contextualized** (ELMo, BERT: context-dependent). **Teacher forcing** (use ground truth) vs. **Scheduled sampling** (gradual transition). **PPO** (4 models, on-policy, advantage-based, complex, more stable for hard tasks) vs. **DPO** (2 models, off-policy, direct preference loss, simpler, may plateau). **REINFORCE** (basic policy gradient, high variance, simple) vs. **PPO** (adds value baseline + KL constraint) vs. **DPO** (bypasses RM entirely). **Greedy** (fast, suboptimal) vs. **Beam** (better, repetitive) vs. **Sampling** (diverse, risky). **Intrinsic** (similarity/analogy) vs. **Extrinsic** (downstream performance) evaluation.